Experiment No : 7

Title: Evaluation of postfix expression (input will be postfix expression).

Name of Student: Shamiksha Chaudhari
Class: SY **Div:** SY1 **PRN No:** B25ET1005 **Batch:** A1

Program:

Postfix evaluation

```
#include <stdio.h>
#include <ctype.h>
#define SIZE 100
char stack[SIZE];
int top = -1;
void push(char c) {
stack[++top] = c;}
char pop() {
return stack[top--];}
char peek() {
return stack[top];}
int isEmpty() {
return (top == -1);}

int precedence(char op) {
if (op == '^') return 3;
if (op == '*' || op == '/') return 2;
if (op == '+' || op == '-') return 1;
return 0;}

void infixToPostfix(char infix[]) {
char postfix[SIZE];
int i, k = 0;
```

```
for (i = 0; infix[i] != '\0'; i++) {
char c = infix[i];

if (isalnum(c)) {
postfix[k++] = c; }

else if (c == '(') {
push(c);
}

else if (c == ')') {
while (!isEmpty() && peek() != '(') {
postfix[k++] = pop();
}
pop();
}
else {
while (!isEmpty() && precedence(peek()) >= precedence(c)) {
postfix[k++] = pop();
} push(c) }
}

while (!isEmpty()) {
postfix[k++] = pop();
}
postfix[k] = '\0'; // Null terminate string
printf("Postfix Expression: %s\n", postfix);
}

int main() {
char infix[SIZE];
printf("Enter Infix Expression: ");
```

```
scanf("%s", infix);
infixToPostfix(infix);
return 0;
}
```

OUTPUT:

